

1. Title

GPU-Accelerated Asynchronous Multi-Stream Object Detection and Behavior Analysis System using CUDA-based Parallel Computing

2. Dataset

Dog Detection Dataset:

<https://universe.roboflow.com/detection-dog/detection-dogs>

Dog Behavior / Pose Dataset:

<https://universe.roboflow.com/project-lgf8z/dddog>

Additional Input:

Multi-camera video streams

Configuration file (camera-wise access timings)

3. Methodology

The system is designed as a GPU-first, CUDA-accelerated asynchronous pipeline, focusing on maximizing parallelism and minimizing data transfer overhead.

Stage 1: GPU-Accelerated Coarse Detection

Video streams are decoded and processed using GPU acceleration.

A YOLO-based model (PyTorch + CUDA + cuDNN) performs detection of:

Dogs

Persons

Parallel inference is achieved across streams using GPU batching.

For each detection:

Metadata is generated (class, bounding box, confidence, timestamp, camera ID)

ROI is extracted using CuPy / Numba-based GPU kernels to avoid CPU bottlenecks

Key Optimization:

Instead of transferring full frames between stages, only ROI tensors remain on GPU memory, reducing PCIe transfer overhead.

Stage 2: Asynchronous GPU Pipelines (Event-Driven Execution)

Detected objects are routed into specialized pipelines using async queues, enabling concurrent execution.

Dog Pipeline (GPU Behavior Analysis)

Input: Dog ROIs (GPU tensors)

Processing:

Pose estimation and behavior inference using PyTorch (CUDA-enabled)

Custom preprocessing accelerated using Numba CUDA kernels

Goal:

Detect aggressive posture / potential bite scenarios

Logging & analytics:

Event logs stored and processed using cuDF (GPU DataFrames) for fast aggregation

Person Pipeline (GPU + Rule-Based Analytics)

Input: Person ROIs + metadata

Processing:

Context-aware validation using camera ID

Time-based access control using config file

If violation detected:

Event logged as unauthorized access

Optimization:

Metadata filtering and joins handled using cuDF for parallel data processing

GPU Acceleration Techniques Used

CUDA & cuDNN → Fast deep learning inference

PyTorch (CUDA backend) → Model execution on GPU

CuPy → GPU-based array operations (NumPy replacement)

Numba (CUDA JIT) → Custom GPU kernels for ROI extraction & preprocessing

cuDF → High-performance GPU DataFrame operations for analytics

Optional: nvJPEG / GPU decoding for faster video frame processing

System Architecture Features

Asynchronous execution using asyncio / multiprocessing

ROI-based data flow to reduce memory bandwidth usage

GPU batching for improved throughput

Multi-stream scalability with efficient GPU utilization

4. Expected Results

High-performance real-time processing using GPU acceleration

Significant reduction in latency due to minimized CPU-GPU transfers

Efficient utilization of GPU cores via batching and parallel streams

Accurate detection of:

Dog aggression / bite risk

Unauthorized human access

Faster analytics using GPU data processing (cuDF)

Demonstration of clear performance improvement over CPU-based or sequential systems

5. Future Scope

Deploy on edge GPU devices (NVIDIA Jetson series)

Integrate real-time alerting systems (SMS, dashboards)

Expand GPU pipelines for more behaviors (violence detection, anomaly detection)

Use TensorRT optimization for faster inference

Implement distributed multi-GPU processing

Integrate streaming frameworks like Kafka for large-scale deployment